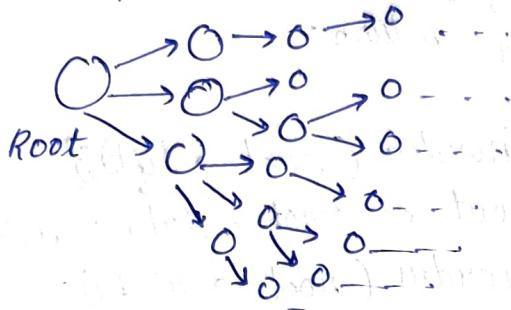


TREE

lc 62

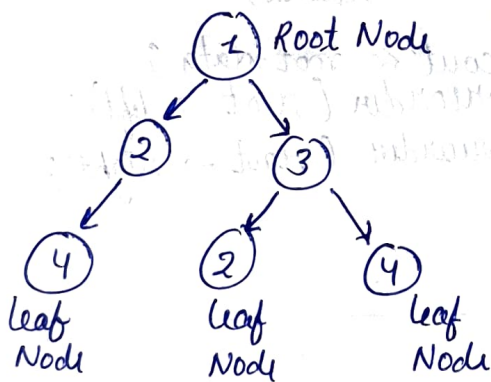
→ Non-linear data structure in which 1 Node can point to multiple Nodes



if 1 node points only 1 Node it's called simple linked-list. (aka skewed tree)

Binary Tree

→ maximum 2 children



```
Node {
    int data;
    int * left;
    int * right;
};
```

Class Node {

```
    int data;
    Node* left;
    Node* right;
}
```

Implementation using Recursion

Class Node {

Public:

```
    int data;
    Node* left;
    Node* right;
```

Node (int d) {

```
    this->data = d;
    this->left = NULL;
    this->right = NULL;
```

};

Node* BuildTree (Node* root) {

```
    cout << "Enter data ";
    int data;
    cin >> data;
```

root = new Node (data);

```
    if (data == -1) {
        return NULL;
    }
```

~~cout << "Enter data left"~~

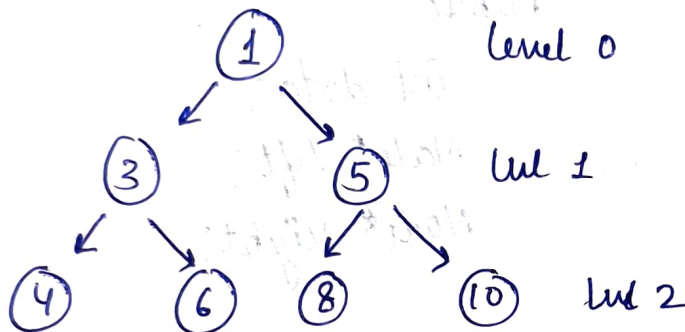
```
    root->left = BuildTree (root->left);
    root->right = BuildTree (root->right);
    return root;
```

```

int main() {
    Node* root = NULL;
    root = BuildTree(root);
    return 0;
}

```

Level Order Traversal



1 3 4 - - 6 - - 5 8 - - 10 - -

Output: 1
3 5
4 6 8 10

1) Inorder / Preorder / Postorder LNR NLR LRN Traversal

1) Inorder
4 3 6 1 8 5 10

2) Preorder
1 3 4 6 5 8 10

3) Post order
4 6 3 8 10 5 1

Codes

1) Inorder LNR

```

void inorder (Node* root) {
    if (root == NULL) {
        return;
    }
    inorder (root -> left);
    cout << root -> data << endl;
    inorder (root -> right);
}

```

2) Preorder NLR

```

void preorder (Node* root) {
    if (root == NULL) {
        return;
    }
    cout << root -> data;
    preorder (root -> left);
    preorder (root -> right);
}

```

3) Post order LRN

```

void postorder (Node* root) {
    if (root == NULL) {
        return;
    }
    postorder (root -> left);
    postorder (root -> right);
    cout << root -> data;
}

```

```
void levelOrder (Node* root) {
```

```
    queue (Node*) q;
```

```
    q.push (root);
```

```
    q.push (NULL);
```

```
    while (!q.empty()) {
```

```
        Node* temp = q.front();
```

```
        q.pop();
```

```
        if (temp == NULL) {
```

```
            cout << endl;
```

```
            if (!q.empty()) {
```

```
                q.push (NULL);
```

```
            }
```

```
        } else {
```

```
            cout << temp->data << " ";
```

```
            if (temp->left != NULL) {
```

```
                q.push (temp->left);
```

```
            }
```

```
            if (temp->right) {
```

```
                q.push (temp->right);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

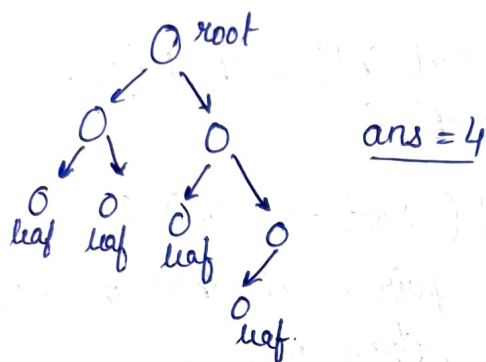
```
return 0;
```


Q.) Create Tree from level order traversal.

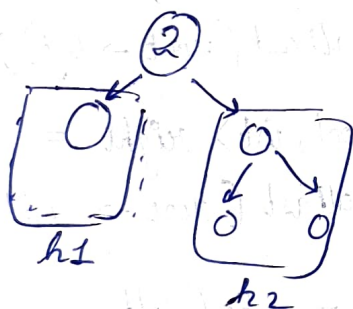
lec 63

Q.1) Find height of Binary Tree.

→ return max^m distance between root Node & leaf Node.



approach: Recursion.



$$\text{ans} = \max(h_1, h_2) + 1$$

int maxHeight (Node* root) {
// base case

~~if (left == NULL && right == NULL)
return 1;~~

if (root == NULL) return 0;

int left = maxHeight (root->left);

int right = maxHeight (root->right);

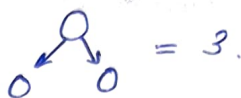
return max (left, right) + 1;

}

$$T.C = O(n) \quad S.C = O(h)$$

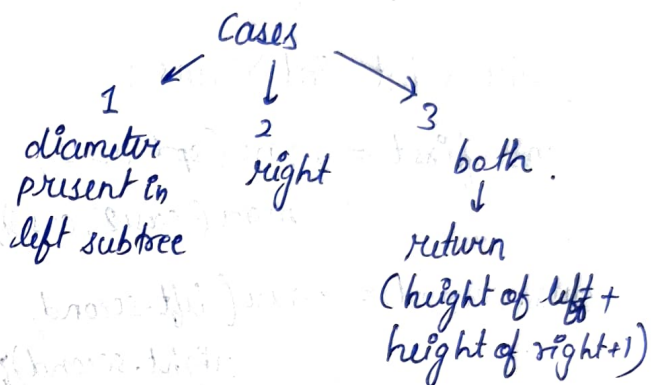
Diameter of Tree

→ longest distance bet^w any two Nodes.



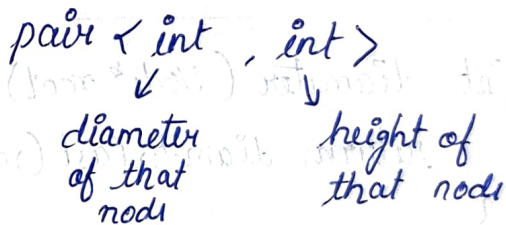
Q.2) Find diameter of a tree

approach 1



$$T.C = O(n^2)$$

approach 2 : using pair <int, int>



pair <int, int> diameterFast (Node* root) {

if (root == NULL) {

pair <int, int> p =

make - pair (0, 0);

return p;

}

```

pair <int, int> left =
    diameterFast(root->left);
pair < > right = diamFast(root->right);

```

```

int case1 = left.first;
int case2 = right.first;
int case3 = left.second,
            right.second;

```

```

pair <int, int> ans;
ans.first = max(case1,
                max(case2, case3));
ans.second = max(left.second,
                 right.second) + 1;

```

```

return ans;
}

```

```

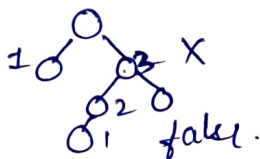
int diameter(Node* root) {
    return diameterFast(root).first;
}

```

Q.3) Check for Balanced Tree.

Balanced Tree

→ diff betⁿ heights of left & right subtree is not greater than 1 for each Node.



approach

3 conditions

- 1) left True
- 2) right True
- 3) $|(left.height) - (right.height)| \leq 1$

code

```

pair <bool, int> balFast(TreeNode*
    root) {
    if (root == NULL) {
        pair <bool, int> p = {true, 0};
        return p;
    }

```

```

    pair <bool, int> left =
        balFast(root->left);

```

```

    pair <bool, int> right =
        balFast(root->right);

```

```

    bool diff = abs(left.second -
                    right.second) > 1;

```

```

    pair <bool, int> ans;

```

```

    ans.second = max(left.second,
                    right.second) + 1;

```

```

    if (left.first && right.first &&
        !diff) {

```

```

        ans.first = true;

```

```

    } else {
        ans.first = false;

```

```

    } return ans;
}

```



```
bool isBal (TreeNode* root) {
    return balFast(root).first;
}
```

Q-4) Check if two trees are identical or Not. (3 conditions)

```
bool isSame (Node* p, Node* q) {
    // base cases
    if (p == NULL && q == NULL) {
        return true;
    }
    if (p != NULL && q == NULL) {
        return false;
    }
    if (p == NULL && q != NULL) {
        return false;
    }
}
```

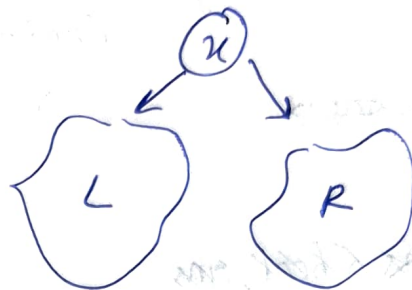
// recursive call.

```
bool left = isSame(p->left, q->left);
bool right = isSame(p->right, q->right);
bool value = p->val == q->val;
bool ans;
if (left && right && ans) {
    return ans = true;
}
else {
    ans = false;
}
return ans;
}
```

T.C = $O(n)$

Q-5) Sum Tree

$(\text{sum of left subtree}) = (\text{sum of right subtree})$



$x = L + R$ for each Node.

3 conditions

- 1) left subtree $\rightarrow T$
- 2) right subtree $\rightarrow T$
- 3) $L + R = \text{root} \rightarrow \text{val}$.

code

```
pair < bool, int > isSumTreeFast
(Node* root) {
```

// base cases

```
if (root == NULL) {
    pair < bool, int > p = {T, 0};
    return p;
}
```

```
if (root->left == NULL &&
    root->right == NULL) {
    pair < bool, int > p = {T, root->val};
    return p;
}
```

pair (bool, int) left = isSumTreeFast (root → left);

pair (bool, int) right = isSumTreeFast (root → right);

~~bool ans;~~

~~if~~

pair (bool, int) ans;

bool condition = root → val ==
left.second + right.second;

~~if~~

pair (bool, int) ans;

if (left.first && right.second &&
condition) {

ans.first = True;

ans.second = 2 * root → data;

} else {

ans.first = false;

}

return ans;

}

bool isSumTree (Node * root) {
return isSumTreeFast (root);
}

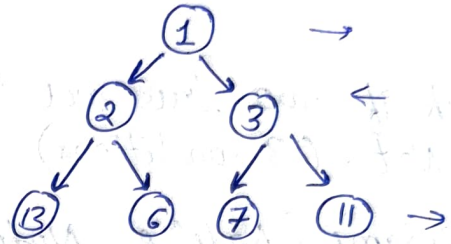
T.C = $O(n)$.

S.C = $O(H)$.

lc 64

Q.1) Zig-Zag Traversal

i/p



o/p 1, 3, 2, 13, 6, 7, 11
using flag & queue.

code:

```

vector (int) zigzag (Node * root) {
    vector (int) result;
    if (root == NULL) {
        return result;
    }
}

```

queue (Node *) q;

q.push (root);

bool leftToRight = True;

while (!q.empty()) {

int size = q.size();

vector (int) ans (size);

// Each level process

for (int i=0; i < size; i++) {

Node * frontNode = q.front();

q.pop();

int index = leftToRight?
i; size-i;


```
ans[index] = frontNode->data;
if (frontNode->left) {
    q.push(frontNode->left);
}
if (frontNode->right) {
    q.push(frontNode->right);
}
```

```
} leftToRight = !leftToRight;
for (auto i : ans) {
    result.push_back(i);
}
return result;
```

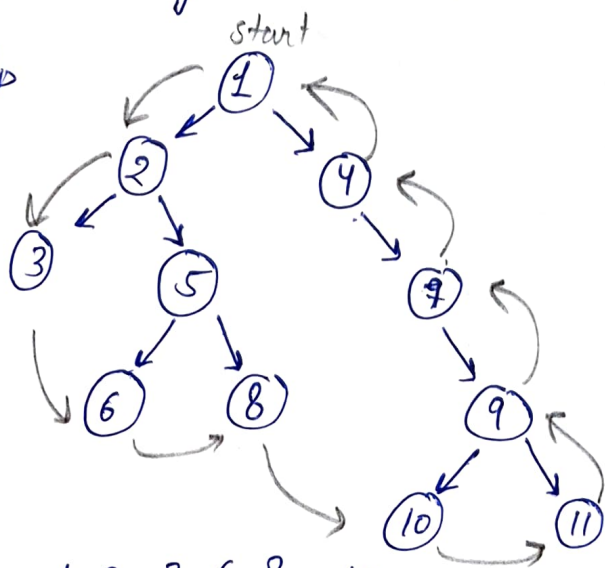
```
};
```

- ① Level process
- ② Direction change
- ③ push temp ans in result.

T.C = $O(n)$ S.C = $O(n)$

Q.2) Boundary Traversal

i/p



o/p: 1, 2, 3, 6, 8, 10

11, 9, 7, 4.

approach

- ① Print left part (No leaf)
- ② Print all leaf nodes
- ③ Print right in reverse order. (No leaf Nodes).

Code

```
void leftTraversal(Node* root,
    vector<int> &ans) {
    if (root == NULL || (leafNode)) {
        return;
    }
```

```
ans.push_back(root->data);
if (root->left) {
    leftTraversal(root->left, ans);
} else {
    leftTraversal(root->right, ans);
}
```

```
}
```

```
void leafTraversal(Node* root,
    vector<int> &ans) {
    if (root == NULL) {
        return;
    }
```

```
if (leafNode) {
    ans.push_back(root->data);
    return;
}
```

```
leafTraversal(root->left, ans);
leafTraversal(root->right, ans);
```

```
}
```



```

void TraverseRight (Node* root, ans){
    if (root == NULL || leafNode){
        return;
    }
    if (root → right){
        TraverseRight (root → right, ans);
    }
    else{
        TraverseRight (root → left, ans);
    }
    ans.push_back (root → data);
}

```

```

vector <int> boundary (root) {
    vector <int> ans;
    if (root == NULL)
        return ans;
    ans.push_back (root → data);
    leftTraversal (root → left, ans);
    leafTraversal (root → left, ans);
    leafTraversal (root → right, ans);
    RightTraversal (root → right, ans);
    return ans;
}

```